Python

The essentials, at a glance.

01 Start here

Install: python.org/downloads

```
python3 --version
python3 app.py
```

Windows: use python or py. Run python3 alone to open the interactive shell.

Use 4 spaces per indent. A # starts a comment.

02 Variables & types

```
name = "Maya"          # str
age = 28               # int
price = 19.95          # float
active = True          # bool
result = None          # no value
```

Use snake_case names. True, False, and None are capitalized. No type declaration is required.

03 Operators & truthiness

`+ - * /`	Arithmetic
`// % **`	Floor, remainder, power
`== != < > <= >=`	Compare values
`and or not`	Boolean operators
`in / not in`	Membership

```
7 / 2     # 3.5
7 // 2    # 3
2 ** 3    # 8
```

False, None, zero, and empty strings or collections are falsy. Check None with is None.

04 Strings

Text is immutable; methods return new values.

```
name = "Maya"
f"Hi, {name}!"       # 'Hi, Maya!'
f"{3.14159:.2f}"     # '3.14'
"  hi  ".strip()     # 'hi'
"Hello".lower()      # 'hello'
"a,b".split(",")     # ['a', 'b']
"-".join(["a", "b"]) # 'a-b'
"cat".replace("c", "b")  # 'bat'
```

05 Lists

Ordered and mutable; duplicates are allowed.

```
nums = [10, 20, 30]
nums.append(40)       # add one
nums.extend([50, 60]) # add many
nums.remove(20)       # first match
last = nums.pop()     # remove last
len(nums)             # item count
nums.sort()           # change nums
ordered = sorted(nums) # new list
```

sort() changes the list and returns None. sorted() returns a new sorted list.

06 Indexing & slicing

```
text = "Python"
text[0]       # 'P'
text[-1]      # 'n'
text[1:4]     # 'yth'
text[::-1]    # 'nohtyP'
```

sequence[start:stop:step]. Start is included; stop is excluded. Works with lists and tuples too.

07 Dictionaries

Mutable key-value mappings. Keys are unique and must be hashable, such as strings or numbers.

```
user = {"name": "Maya", "age": 28}
user["name"]              # 'Maya'
user["age"] = 29
user.get("role", "guest") # fallback
"name" in user            # True
for key, value in user.items():
    print(key, value)
```

d[key] raises KeyError for a missing key. get() reads a fallback without adding the key.

08 Tuples & sets

```
point = (10, 20)
x, y = point        # unpack
single = (42,)      # one-item tuple
tags = {"web", "api", "web"}
tags.add("python")
tags.discard("api") # safe if absent
{1, 2} | {2, 3}     # union: {1, 2, 3}
{1, 2} & {2, 3}     # intersection: {2}
```

Tuples are immutable. Sets hold unique, hashable values in no guaranteed order. Empty set: set().

09 Comprehensions

```
nums = [1, 2, 3, 4]
squares = [n * n for n in nums]
evens = [n for n in nums if n % 2 == 0]
unique = {n % 2 for n in nums}
```

Use [ ] for a list, {key: value} for a dict, and {value} for a set. Add if to filter.

Python

Everyday patterns, ready to use.

10 Conditionals

```
score = 82
if score >= 90:
    grade = "A"
elif score >= 80:
    grade = "B"
else:
    grade = "C"
```

11 Loops

```
for n in range(3):
    print(n)        # 0, then 1, then 2
names = ["Maya", "Leo"]
for i, name in enumerate(names, start=1):
    print(i, name)

count = 3
while count > 0:
    count -= 1
```

range(start, stop, step) excludes stop. break exits the loop; continue skips to the next iteration.

12 Functions

```
def greet(name, greeting="Hello"):
    return f"{greeting}, {name}!"

greet("Maya")
greet("Leo", greeting="Hi")
```

*args collects positional arguments; **kwargs collects keyword arguments. Without return, a function returns None.

13 Input & exceptions

input() returns a string. Convert it when you need a number, and handle invalid input.

```
try:
    age = int(input("Age: "))
except ValueError:
    print("Enter a whole number.")
else:
    print(f"Next year: {age + 1}")
```

Also: float("3.5") converts to a float; str(42) converts to text. Catch specific errors.

14 Files & JSON

```
from pathlib import Path
import json

path = Path("notes.txt")
path.write_text("Hi!", encoding="utf-8")
text = path.read_text(encoding="utf-8")
user = {"name": "Maya", "active": True}
payload = json.dumps(user)  # to JSON
data = json.loads(payload) # to Python
```

write_text() replaces existing contents. Relative paths start from the current working directory.

15 Useful built-ins

`len(items)`	Number of items
`sum(numbers)`	Add numeric values
`min(x) / max(x)`	Smallest / largest item
`any(x) / all(x)`	Any / all items truthy
`zip(a, b)`	Pair items; shortest wins

16 Classes

```
class Task:
    def __init__(self, title):
        self.title = title
        self.done = False
    def complete(self):
        self.done = True
task = Task("Learn Python")
task.complete()
print(task.done)     # True
```

__init__ initializes an instance. self refers to that instance inside its methods.

17 Virtual environments

```
# macOS / Linux (bash or zsh)
python3 -m venv .venv
source .venv/bin/activate
# Windows (Command Prompt)
python -m venv .venv
.venv\Scripts\activate.bat
# After activation
python -m pip install requests
deactivate
```

Each project gets its own packages. Keep .venv out of version control.

18 Remember these

`== versus is`	Equal values vs. same object
`a = b`	Shares an object; no copy
`[] as a default`	Shared between function calls

For a fresh list per call, default to None and create the list inside the function.